CODEFORCES 2034C

C. Trapped in the Witch's Labyrinth

Proof-oriented solution notes · Functional graph · DFS · Cycle detection

Problem statement: https://codeforces.com/contest/2034/problem/C

Contest: Codeforces Round 989 (Div. 1 + Div. 2), Problem C. These notes explain the accepted C++ implementation supplied alongside the request.

Objective. Choose directions for all '?' cells so that the number of starting cells whose trajectories never leave the grid is maximized.

1. Mathematical model

Represent every cell by a vertex. A fixed arrow gives that vertex one outgoing edge to the indicated neighbour. An edge leaving the grid leads to an external sink, denoted OUT. After assigning every '?', each in-grid vertex has exactly one outgoing edge.

Thus, the completed maze is a **functional directed graph**: every vertex has outdegree exactly one (if OUT is included, OUT is an absorbing escape state).

Lemma 1. From any starting vertex, exactly one of the following happens: (i) the trajectory reaches OUT, or (ii) it eventually repeats a vertex and hence enters a directed cycle.

Proof. The successor of every in-grid vertex is uniquely determined. If the trajectory never reaches OUT, it is an infinite sequence of vertices drawn from a finite set of nm vertices. By the pigeonhole principle, some vertex occurs twice. From the first repeated occurrence onward, determinism forces the same successor sequence, producing a cycle. Conversely, once a cycle is reached, the trajectory repeats it forever. ■

Call a cell **good** if it reaches OUT and **bad** if it reaches a cycle. The program encodes good = 0 and bad = 1.

2. State and invariant

```
bad[i][j] = -1   not classified yet
bad[i][j] =  0   good: the trajectory can be made to exit
bad[i][j] =  1   bad: the trajectory can be made to stay forever

instack[i][j] = 1 iff (i,j) is on the current recursive DFS path
```

The arrays have an extra one-cell border. Every border entry of `bad` is initially 0. The border represents OUT, so a fixed arrow leaving the grid immediately obtains result 0.

DFS invariant. While `f(i,j)` is executing, every cell marked in `instack` belongs to the current directed path from the original DFS root to the current cell. A non--1 value in `bad` is a classification already established and can be reused.

3. Classifying cells with fixed arrows

```
int f(int i, int j) {
    if (instack[i][j]) return 1;
    if (bad[i][j] != -1) return bad[i][j];
    if (a[i][j] == '?') return bad[i][j] = 1;

    instack[i][j] = 1;
    // Recursively follow the unique arrow:
    // U -> (i-1,j), D -> (i+1,j)
    // L -> (i,j-1), R -> (i,j+1)
    bad[i][j] = f(successor(i,j));
    instack[i][j] = 0;
    return bad[i][j];
}
```

Why does an active-stack revisit return 1?

Lemma 2. If DFS reaches a vertex already marked in `instack`, the current trajectory has entered a directed cycle.

Proof. The active vertex was visited earlier on this same recursive path. Since each fixed cell has one deterministic successor, the segment of the path between the two occurrences forms a directed cycle. Therefore the current cell and every predecessor on the recursion path eventually enter that cycle. Returning 1 is correct. ■

Why does the result propagate to predecessors?

Every fixed cell has exactly one successor. If its successor is bad, then the cell itself follows that successor and is also bad. If its successor is good, the cell follows it out and is good. Therefore copying the recursive result to the current cell is logically valid.

Why is '?' provisionally assigned 1 here?

A fixed-arrow DFS reaches '?' only after following a fixed path into that unspecified cell. Let v be the predecessor on this path and q the reached '?'. We may assign q to point back to v. This closes a directed cycle containing q and v; all earlier cells on the path then feed into that cycle. Hence every fixed cell on this DFS path can be made bad. The value 1 is therefore the correct maximum-achievable classification for this DFS path.

Memoization: Once `bad[v]` is 0 or 1, every later path reaching v has the same fate under the chosen optimal completion, so DFS can return the stored result without exploring again.

4. The remaining '?' cells

After all fixed-direction cells have been started in DFS, any cell still having `bad = -1` must be an unspecified cell not reached by any fixed-arrow path. Consider the graph induced by these unresolved '?' cells, where two cells are adjacent if they share a side.

Lemma 3. Every connected component of unresolved '?' cells containing at least two vertices can be assigned directions so that all its vertices are bad.

Proof. A connected component with at least two vertices contains an edge (u,v). Assign u→v and v→u, forming a directed 2-cycle. Since the component is connected, choose a spanning tree rooted at u. Direct every other vertex along the unique tree path toward u. Every vertex then eventually reaches the 2-cycle, so all are bad. ■

Lemma 4. An isolated unresolved '?' cell q is bad if and only if at least one of its four neighbouring cells is already bad.

Proof. If a neighbour is bad, point q toward that neighbour; q then reaches a cycle and is bad. If all in-grid neighbours are good (and any outside neighbour represents immediate escape), every possible direction from q leads to escape. Since q has no unresolved '?' neighbour, it cannot participate in a cycle with another unassigned cell. Thus q must be good. ■

5. Translating the proof into the second pass

```
if (bad[i][j] == -1) {
    int x = 0;
    for (each of the four neighbours v) {
        if (bad[v] == -1)
            x = 1;             // adjacent unresolved '?'
        else
            x |= bad[v];       // can point to a bad neighbour
    }
    bad[i][j] = x;
}
```

For an unresolved cell, `bad[neighbour] == -1` means that neighbour is another unresolved '?'. By Lemma 3, this cell belongs to a non-singleton unresolved component and can be made bad. If there is no such neighbour, the cell is isolated in the unresolved-'?' graph; Lemma 4 says it is bad exactly when at least one neighbour is already bad.

Because states are bits 0 and 1, bitwise OR computes whether any inspected neighbour is bad. The padded border has value 0, correctly representing an outside move.

6. Correctness theorem

Theorem. The algorithm outputs the maximum possible number of trapped starting cells.

Proof. Partition the cells into two groups after the DFS phase.

Group A: fixed-arrow cells. Each such cell has a unique successor. By Lemmas 1 and 2 and the successor propagation argument, DFS classifies it according to whether its forced trajectory exits or enters a cycle. If its forced path first reaches an unresolved '?', that '?' can point back along the path, making the whole path trapped; assigning it 1 is therefore achievable. Memoization preserves this classification for every later path reaching the same cell.

Group B: unresolved '?'-cells. Their connected components are independent in the unresolved-'?' adjacency graph. Every component of size at least two can be made entirely trapped by Lemma 3. A singleton component is trapped exactly when it has a neighbouring already-bad cell, by Lemma 4. The second pass assigns precisely these optimal values.

The two groups cover all cells, and every assigned 1 is simultaneously achievable: non-singleton '?' components can be oriented toward their own 2-cycle, isolated '?' cells can point toward a bad neighbour, and fixed paths reaching '?' can be closed into cycles. Every assigned 0 is unavoidable under any completion. Hence the sum of `bad[i][j]` is exactly the maximum number of trapped starting cells. ■

7. Complexity

Measure	Bound	Justification
Time	**O(nm)**	Each fixed cell is classified once; each unresolved '?' inspects four neighbours.
Space	**O(nm)**	Grid and two state matrices; recursive stack is O(nm) in the worst case.

The padded grid uses O((n+2)(m+2)) memory, which is O(nm). The recursive implementation may reach depth O(nm); an iterative traversal can be used if the execution environment has a restrictive stack limit.

8. Sample results

Sample 1 → **0.** The fixed arrows lead out; no starting cell can be made trapped.

Sample 2 → **6.** All six cells are unspecified. The grid is connected and has more than one vertex, so the whole component can be oriented toward a 2-cycle.

Sample 3 → **5.** The fixed directions and unresolved '?' components yield five cells that can be made trapped; the remaining cell is forced to escape.

9. What to remember

• A deterministic successor relation gives a functional graph.
• A finite functional-graph trajectory either exits or eventually cycles.
• DFS recursion-stack membership detects a cycle; memoization prevents repeated work.
• A connected unresolved-'?' component with at least two cells can be routed into a 2-cycle.
• An isolated unresolved '?' is trapped exactly when it can point to an already-trapped neighbour.

Suggested repository path: `Graph-Theory/Functional-Graphs/Codeforces-2034C/`
Suggested files: `solution.cpp` and `explanation.pdf`